M

Functioneel Programmeren

Authors

Matt ter Steege

Matttersteege@gmail.com

November 9, 2026

Contents

1 Introductie tot Haskell . . . 3
1.1 Lijsten . . . 4
1.2 Tuples . . . 4
1.3 Guards . . . 4
1.4 Lokale definities . . . 4
1.5 pattern matching . . . 5
1.6 Types . . . 5
2 Lijsten . . . 6
2.1 Recursie . . . 6
2.2 List comprehensions . . . 6
3 Data types . . . 6
4 Higher-order functions . . . 7
4.1 map en filter . . . 8
4.2 Anonieme functies . . . 8
4.3 Currying en partial application . . . 8
4.4 Function composition . . . 9
4.5 Folds . . . 9
5 Purely functional data structures . . . 10
5.1 Persistent versus ephemeral . . . 10
5.2 Successor datastructuur . . . 10
5.3 Red-black trees . . . 10
6 Case studies . . . 11
6.1 Proposities . . . 11
6.2 Rekenkundige expressies . . . 12
7 Software design en project management . . . 13
7.1 Modules en packages . . . 13
7.2 Software design . . . 13
8 Input en output . . . 14
8.1 Purity en side effects . . . 14
8.2 Een model van IO . . . 14
8.3 do-notatie . . . 15
8.4 IO als first-class citizen . . . 15
8.5 Randomness . . . 16
9 Functors en monads . . . 16
9.1 Functors . . . 16

9.2 Monads . . . 16
9.3 De list monad . . . 17
10 Wetten en inductie . . . 17
10.1 Wetten en equational reasoning . . . 17
10.2 Welk bewijsprincipe? . . . 18
10.3 Structurele inductie . . . 18
11 Lazy evaluation . . . 19
11.1 Evaluatiestrategieën . . . 19
11.2 Waarom is lazy handig? . . . 19
11.3 Weak Head Normal Form . . . 19
11.4 Lazy en performance . . . 20
12 Property testing met QuickCheck . . . 20
12.1 Case study: insertion sort . . . 20
12.2 QuickCheck in het algemeen . . . 21
12.3 Eigen generators . . . 21
12.4 Verder dan testen . . . 21
13 State monad en applicatives . . . 22
13.1 De State monad . . . 22
13.2 Monad wetten . . . 22
13.3 Applicatives . . . 23

Probeer de online Haskell compiler online: https://play.haskell.org

1 | Introductie tot Haskell

Functioneel programmeren (FP) gebruikt vaak recursie in plaats van loops. In plaats van for- en while-loops laat je een functie zichzelf herhalen totdat een basisgeval is bereikt. Pattern matching betekent dat je data direct op zijn vorm controleert, bijvoorbeeld leeg of niet-leeg. Je vervangt daardoor veel if/else-constructies door duidelijke gevallen per datastructuur. In FP werk je vooral met expressies die een waarde teruggeven, in plaats van statements die alleen acties uitvoeren. Functies zijn "first-class citizens", wat betekent dat je ze kunt doorgeven, opslaan en teruggeven zoals gewone waarden. Daardoor kun je functies combineren tot nieuwe functies en wordt code meer modulair opgebouwd.

Een kort voorbeeld van Haskell:

Code:

```
1 sumUpTo :: Int → Int
2 sumUpTo 0 = 0
3 sumUpTo n = n + sumUpTo (n-1)
4
5 main = print (sumUpTo 3)
```

Output:

```
6
```

Maar wat gebeurt hier nou? Bekijk het zo: in regel 4 `main = print (sumUpTo 3)` wordt een function aanroep gedaan en main wordt hier gelijk aan gesteld. het C# equivalent zou zo zijn: `Console.WriteLine(sumUpTo(3))`. Nou zie je 2 verschillende functies van `sumUpTo`, de interpreter kijkt van boven naar beneden en zoek de eerste passende functie. de parameter "3" voldoent niet aan de case "0", maar wel aan de case "n". Dus de uitkomst van `sumUpTo 3 = 3 + sumUpTo 2`. Dit blijft zich herhalen totdat de basecase "0" wordt bereikt, want deze geeft heeft geen functie in zijn definitie. Het volledige genomen pad gaat dus als volgt:

$$\begin{aligned}\text{sumUpTo } 3 &\to 3 + \text{sumUpTo } 2 \\ &\to 3 + 2 + \text{sumUpTo } 1 \\ &\to 3 + 2 + 1 + \text{sumUpTo } 0 \\ &\to 3 + 2 + 1 + 0\end{aligned}$$

Type signature

Dan moet ik alleen de eerste regel nog uitleggen: `sumUpTo :: Int → Int`, deze regel beschrijft de "Type signature". Dit geeft aan hoe de functie aangeroepen moet worden. De laatste type geeft aan wat de return type wordt (`Int`) en wat de parameters zijn (`Int`). Stel nou dat we dit hebben: `sumUpTo :: Int → Bool → String → Int`, dan hebben we dus 3 parameters, een `Int`, `Bool` & `String` en de return type is nog steeds `Int`. Maar dit staat nog niet helemaal vast, dat ga ik in de volgende subsectie nog even extra uitleggen.

Type variabeles

Types zijn op verschillende manieren te definiëren, je hebt de standaard `Int`, `Bool`, `String`, ..., maar je hebt ook "type variabeles" dit zijn types die niet specifiek gedefineerd zijn. Neem het voorbeeld `replicate Int → a → [a]`. De eerste parameter is van het type `Int`, de tweede parameter is van type `a`, wat het type van de aanroep is maakt niets uit, dit kan elke type zijn wat je wilt. De return value is een list van type a. Dus als je de aanroep doet: `replicate 10 "replicate"`, dan krijg je een string list terug. De interpreter zegt dan eigenlijk: "in deze aanroep lezen we type `a` als `String`"

Lijsten

list worden op de volgende manier genoteerd: `[1, 2, 3]` of `[1 .. 5]` (een list met de waarde 1 t/m 5). Basis functies zijn `length [1 .. 5]` is 5. Functies zoals `sum [Int] → Int`, `reverse [a] → [a]`, en `replicate Int → a → [a]` zijn functies die wel redelijk logisch zijn.

Wat ik nu ga zeggen is super wild, maar [] is een lege lijst, hierin zitten geen waarden en heeft lengte 0. Als je hier nou later een waarde aan wilt toevoegen, dan doe je dat door middel van `1 : [] = [1]` of `2 : [1] = [2,1]` hierop kan je verschillende basisfuncties uitvoeren:

```
null [1,2,3] -- Is de list leeg?
> False
head [1,2,3] -- Geef de eerste waarde van de array terug
> 1
tail [1,2,3] -- Geef alle de list zonder de eerste waarde terug.
> [2,3]
```

Lijsten hebben een onbekende lengte, maar wel een uniform type

Tuples

Tuples daarentegen hebben een bekende lengte, maar bestaan (waarschijnlijk) uit verschillende types. Je moet bij het gebruik van tuples bedenken dat deze nogsteeds één object zijn. Je behandeld ze dus hetzelfde als een `Int` oid.

Maken van Tuples:

```
remainder :: Double → (Int,Double)
remainder x = let i = floor x
              in (i, x - fromIntegral i)
{- remainder 6.56 -→ (6, 0.56)
   Je krijgt dus een object terug met 2
   waardes een Int en een Double -}
```

Extraheren van Tuples:

```
dist (px, py) (qx,qy) = sqrt (xDiff + yDiff)
    where
        tpl = squareBoth (px - qx, py - qy)
        squareBoth (xD,yD) = (xD*xD, yD*yD)
        (xDiff, yDiff) = tpl
```

De rechter ga ik een beetje toelichten. Stel dat je `squareBoth (3,4) = (9,16)`. Dan wordt `tpl = (9,16)` *--nog steeds 1 object*. Dan onderaan wordt `(xDiff, yDiff) = (9,16)`. Nu kan je xDiff en yDiff vrij gebruiken als losse functies.

Guards

Hoewel Haskell if statements heeft in het format `if condition then expression else expression` moet je dit gelijk vergeten, dit schaalt heel slecht, daarom gebruiken we guards ("|"). Deze bereiken hetzelfde als If statements, maar lezen veel beter, ze zijn te vergelijken met de switch statement in C#. Guards werken natuurlijker, je gaat namelijk weer van boven naar beneden, de eerste check die voldoet geeft aan wat de return waarde is. Bij n = -0.3 is de return waarde -1. De tweede is veel makkelijker te lezen!

If:

```
sign n = if n < 0
             then -1
             else if n == 0
                 then 0
                 else 1
```

Guards:

```
sign n | n < 0     = -1
       | n == 0    =  0
       | otherwise =  1

-- kijk eens naar deze lege ruimte :0
```

Lokale definities

Ook dit zijn weer functies. Deze kan je in een andere functie definiëren en hebben toegang tot de variabelen in die parent-functie. Hieronder zijn 2 voorbeelden die in werking hetzelfde zijn. Deze functies zijn er om code-repetitie te verminderen en leesbaarheid te vergroten.

Where:

```
distance px py qx qy = sqrt (xDiff + yDiff)
    where
        xDiff = square (px - qx)
        yDiff = square (py - qy)
        square z = z * z

-- Dit gebruik je als je van boven naar beneden
      denkt
```

Let in:

```
distance px py qx qy =
    let xDiff = square (px - qx)
        yDiff = square (py - qy)
        square z = z * z
in sqrt (xDiff + yDiff)

-- Dit gebruik je als je van beneden naar boven
      denkt.
```

Je zegt namelijk de dat de functie xDiff (zonder parameters) gelijk is aan de functie square met de waarde $(p_x - q_x)$. En elke keer als je xDiff "aanroept" krijg je dus die waarde. Het makkelijke hiervan is dat je geen parameters mee hoeft te geven, want de waardes die je wilt gebruiken (px, qx, ...) zijn al beschikbaar in de scope van de lokale functie. Let wel op dat de functies **wél** gevoelig zijn voor uitlijning (zelfde kolom, dus 2 tab's).

pattern matching

Pattern matching is het systeem dat je gebruik bij functie-aanroepen. Het kijk van boven naar beneden en zoekt naar de juiste functie voor de aanroep.

Lang geschreven:

```
conj :: Bool → Bool → Bool
conj True True = True
conj True False = False
conj False True = False
conj False False = False
```

Kort geschreven:

```
conj :: Bool → Bool → Bool
conj True True = True
conj _ _ = False
-- Gebruik "_" als je niets geeft om de waarde.
-- Je zegt dus eigenlijk, True True → True,
     anders False.
```

Types

Een type is een collectie van gerelateerde waardes. En elke expressie heeft een type, deze wordt geschreven als :: type, bijvoorbeeld (`True :: Bool`, `'a' :: char`, `[1, 2] :: [Int]`. Dit geldt ook voor functies zoals `1 + 2 :: Int`.

Basic types zijn `Bool`, `Char`, `Int`, `Integer`, `Float`, `Double`. Dan heb je nog compound types, deze zijn samengesteld. Denk hierbij aan lists en Tuples (met een maximale arity van 62).

Inferring the type of ...

Als je een type wilt afleiden, dan volg je het volgende stappenplan.
1. Zorg dat er geen overlap zit tussen de type variabelen
2. Als `f :: A → B`, in `f e` moet `e :: A`
3. Het resultaat van `f e :: B`

Voorbeeld (`map id ::`):

Gegeven: `map :: (a → b) → [a] → [b]` en `id :: c → c`. `map` verwacht als eerste argument een functie van type `a → b`. Omdat we `id` doorgeven, moeten deze types gelijk zijn: $a \to b = c \to c$. Dus volgt:
$a = c$ $b = c$. Vervang deze in het type: `map :: (c → c) → c → c` Het eerste argument (`id`) is nu ingevuld, dus blijft over: `map id :: [c] → [c]`

Type classes

type classes zijn een manier om aan te geven dat een functie alleen op een bepaalde groep types werkt. Dus bijvoorbeeld de type class "Num" is een overkoepelende groep voor Int, Interger, Double en Float. Dit is handig voor functies die alleen werken voor een bepaalde groep. De functie `(+) :: Num a ⇒ a → a → a` (de + operator) werkt alleen op types die Num zijn. Dat is ook de `Num a ⇒`, deze geeft aan dat de type variable a op zijn minst een numerieke waarde moet zijn.

Basis type classes zijn: `Num` voor numerieke types, `Eq` voor types die gelijkheids checks ondersteunen, `Ord` voor types die is een bepaalde volgorde hebben. Int's hebben een volgorde, namelijk $0, 1, 2, 3, 4, \ldots$, als je een

volgorde hebt, dan kan automatisch dingen gebruiken zoals > en <=, omdat je kan checken wat de posities in volgorde zijn tussen de waarde voor en naar de <. `Show` om dingen om te zetten in strings. En er zijn er nog veel meer.

2 | Lijsten

Recursie

Om recursie te begrijpen, moet je eerst recursie begrijpen. Een Haskell voorbeeld is:

```
fac 0 = 1
fac n = n * fac (n - 1) -- Roept zichzelf aan.
```

Hutton's recept om een recursieve functie te schrijven bestaat uit vijf stappen:
1. Definieer het type van de functie.
2. Bepaal alle mogelijke gevallen (patterns).
3. Definieer de basisgevallen.
4. Definieer de recursieve gevallen door het probleem te verkleinen.
5. Vereenvoudig en generaliseer de definitie.

laten we een voorbeeld doorlopen met de sum functie:

Stap:
- Definieer het type: `sum :: [Int] → Int`
- Alle cases: `sum [] = _` en `sum (x:xs) = _`
- De base case: `sum [] = 0`
- De andere case(s) `sum (x:xs) = x + sum xs`
- Vereenvoudig `sum :: Num a ⇒ [a] → a`

Wat doet het

```
<- List van Int's erin, Int eruit
<- lege list, of niet-lege list
<- termineert met een lege list
<- anders voorste element + sum van de rest
<- uiteindelijke type Num a (Int, Float, Double,...)
```

List comprehensions

In Haskell is er de `[expr | x ← list]` notatie. Deze notatie bouwt een nieuwe lijst van een oude lijst.

één extra:

```
addone :: Num a ⇒ [a] → [a]
addone xs = [x + 1 | x ← xs]
```

Deze functie zegt eigenlijk hetzelfde als "Voor elke x in xs, return x + 1". Elke waarde in de lijst word dus één groter. **Som van even:**

```
-- Check is a number is divisible by 2
even :: Integer → Bool

sumeven :: [Integer] → Integer
sumeven xs = sum [x | x ← xs, even x]
```

Deze functie `[x | x ← xs, even x]` zegt het volgende: maak een array van elke x in xs, waar `even x == true`. Dus deze list comprehension zet een lijst zoals `[1, 3, 4, 6, 7]` eerst om in een de lijst `[4,6]` en daarna roept hij `sum` erop, wat uitkomt op 10.

3 | Data types

Stel nou dat je zelf een datatype wilt maken, dan kan je dat zo doen:

```
data Direction = North
               | South
               | East
               | West
```

Het keyword `data` geeft aan dat er een nieuw type gemaakt wordt. De naam van het type "Direction" moet beginnen met een hoofdletter. En dan één of meerdere constructors achter de "=" gesplits door "|". Stel nou dat je `> :t North` doet om het type van North te krijgen, dan krijg je dat North van het type Direction is (idem met lists van North/South/...).
Hier kan je vervolgens ook functies voor schrijven:

```haskell
directionName :: Direction → String
directionName North = "N"
directionName South = "S"
directionName East = "E"
directionName West = "W"

--directionName North → "N"
```

Zo zijn dit voorbeelden van ingebouwde data types:

Bool:
```haskell
data Bool = False | True
```

Int:
```haskell
data Int = ... -1 | 0 | 1 ...
```

Char:
```haskell
data Char = ... 'A' | 'B' ...
```

Echter worden de "oneindige" types (Int, Char,...) wel apart behandeld door de compiler, maar dit is het idee er achter.

maar deze data types slaan inhoudelijk geen data op, deze geven alleen aan wat ze zijn. Om data op te slaan kan je een data type zoals "Point" maken: `data Point = Pt Float Float`. Dit moet je lezen als: Maak een nieuw dataype `Point`. Om een Point te maken gebruik je de constructor `Pt` en deze heeft 2 Floats nodig. Een voorbeeld zou zijn `Pt 2.0 5.0`. (Je had ook `data Point = Point Float Float` mogen schrijven, maar Pt is korter). Wat nou als je de afstand van de oorsprong $(0, 0)$ wilt berekenen, dan kan je dat met een functie zoals dit doen:

```haskell
norm :: Point → Float
norm (Pt x y) = sqrt (x*x + y*y)
```

Je gebruikt hier `(Pt x y)` omdat dat de vorm is die je hebt gedefinieerd hiervoor. Je moet het wel tussen haakjes zetten, omdat dit één type/argument is en niet meerdere.

Zo'n constructor is eigenlijk gewoon óók een functie (wat een mindfuck), want je zou het namelijk ook zo kunnen lezen: `Pt :: Float → Float → Point`. En als je dan met precies dezelfde syntax deze functie aanroept: `Pt 2 5` krijg je dus weer een punt met coordinaten 2 & 5.

Maar het kan nog interessanter. Want één datatype kan meerdere constructors hebben, zoals:

```haskell
data Shape = Rectangle Point Float Float -- linksboven, breedte, hoogte
           | Circle Point Float          -- Midden, Radius
           | Triangle Point Point Point  -- Hoek A, Hoek B, Hoek C
```

Hier kan een "Shape" dus 3 verschillende vormen voorstellen. Dit heet een Sum type (of variant type). Deze variant types kan je vervolgens dmv pattern matching ook weel uit elkaar halen bij een functie. Elke vorm heeft namelijk een oppervlakte (ga ik even gemakshalve vanuit). Dus dan kan je een fuctie schrijven die een Shape als argument geeft en een Float terug geeft.

```haskell
area :: Shape → Float
area (Rectangle _ w h) = w * h -- Wanneer shape een rectangle is (hoekpunt boeit niet)
area (Circle _ r) = pi * r ^ 2 -- Cirkel, midden boeit niet
area (Triangle x y z) = ...    -- Driehoek, iets te lange formule
```

Datatypes kunnen zichzelf ook hebben in de constructor: `data IntTree = EmptyTree | Node Int IntTree IntTree`. Dan is het wel nodig om een EmptyTree te declareren, want anders kan je nooit een eindige boom maken.

4 | Higher-order functions

Puur functioneel programmeren wil code makkelijk te beredeneren maken: data stroomt alleen via argumenten en returnwaardes, er is geen verborgen state en er zijn geen loops, `break` of `goto`. In plaats van loops gebruik je **higher-order functions**: functies die andere functies gebruiken. Dit geeft veel abstractiekracht en dus meer hergebruik van code.

Let op het verschil tussen *curried* en *higher-order*: `f :: a → b → c` is hetzelfde als `f :: a → (b → c)` (curried, meerdere argumenten), terwijl `f :: (a → b) → c` een functie als argument heeft (higher-order). Wat kan je met zo'n argument-functie? Alleen **toepassen**! Je kan er niet op pattern matchen en niet in de broncode kijken, alleen naar het input-output gedrag.

map en filter

`map` past een functie uniform toe op een lijst, `filter` houdt alleen de elementen over die voldoen aan een predicaat (een functie die een `Bool` teruggeeft).

Code:

```
map :: (a → b) → [a] → [b]
map f []     = []
map f (x:xs) = f x : map f xs

filter :: (a → Bool) → [a] → [a]
filter p [] = []
filter p (x:xs)
  | p x       = x : filter p xs
  | otherwise = filter p xs
```

Output:

```
> map length ["a", "abc", "ab"]
[1,3,2]
> filter even [1 .. 4]
[2,4]
> filter (> 10) [1 .. 4]
[]
```

Het maakt niet uit of de functie die je gebruikt globaal is of een argument. Beide zijn ook uit te drukken met list comprehensions: `map f xs = [f x | x ← xs]` en `filter p xs = [x | x ← xs, p x]`. De recursieve definities zijn alleen beter om over te redeneren.

Anonieme functies

Een kleine functie die je maar één keer nodig hebt hoeft geen naam. Met `\x → 2 * x` definieer je een anonieme functie (lambda). De `\` lijkt op een Griekse λ, want de theorie achter FP is het λ-calculus (Alonzo Church, jaren '30). Het is gewoon een functie, dus je kan hem overal gebruiken waar een functie nodig is.

Code:

```
doubleList xs = map (\x → 2 * x) xs

> (\x → 2 * x) 3
> filter (\x → x > 10) [1 .. 20]
```

Output:

```
6
[11,12,13,14,15,16,17,18,19,20]
```

Currying en partial application

In Haskell neemt elke functie **één argument tegelijk**, het resultaat kan weer een functie zijn. Haakjes in functietypes associëren naar rechts. Alle onderstaande definities van `addThree` zijn dus hetzelfde:

```
addThree :: Int → Int → Int → Int      -- = Int → (Int → (Int → Int))
addThree x y z = x + y + z
addThree x y   = \z → x + y + z
addThree x     = \y → \z → x + y + z
addThree       = \x y z → x + y + z
```

Hierdoor kan je ook **minder** argumenten geven dan in de signature staat (partial application). Het resultaat is een nieuwe functie. Dit maakt *eta-reductie* mogelijk: laat het argument aan beide kanten weg.

Code:

```
> :t map (\x → 2 * x)
-- Num b ⇒ [b] → [b]

doubleList xs = map (\x → 2 * x) xs
doubleList    = map (\x → 2 * x)
```

Sections (operators gedeeltelijk toepassen):

```
(x #) = \y → x # y
(# y) = \x → x # y

doubleList   = map (2 *)
largerThan10 = filter (> 10)
-- let op de volgorde!
filter (10 >) [1 .. 20] -- [1..9]
```

Een functie kan ook een functie **teruggeven**. Zo'n functie die andere functies maakt heet een combinator, bijvoorbeeld `flip f = \y x → f x y`, waarmee `flip map [1,2,3] (\x → 2 * x)` gelijk is aan `[2,4,6]`.

Voorbeeld **(applyAll: een lijst van functies)**:

`applyAll [(+ 1), (* 2), (\x → x - 3)] 3` geeft $((3+1)\cdot 2)-3=5$. De basecase is niet `[f]`, maar de lege lijst (denk goed na over de basecase!):

```haskell
applyAll :: [a → a] → a → a
applyAll []     x = x
applyAll (f:fs) x = applyAll fs (f x)
```

Function composition

Nog een combinator: g *na* f. Eerst `f`, dan `g`.

```haskell
(.) :: (b → c) → (a → b) → (a → c)
g . f = \x → g (f x)

odd = not . even                          -- i.p.v. odd x = not (even x)
filterNot p = filter (not . p)            -- verwijder alles wat aan p voldoet

maxAverage :: [[Float]] → Float
maxAverage = maximum . map average . filter (not . null)
  where average xs = sum xs / fromIntegral (length xs)
```

Zo'n pipeline heet *point-free*. Het moet de data flow verduidelijken, niet verbergen. Let op de volgorde: `map (+ 1) . filter even` is hetzelfde als `filter odd . map (+ 1)`, maar niet als `filter even . map (+ 1)`. Ook `applyAll` kan point-free: `applyAll [] = id` en `applyAll (f:fs) = applyAll fs . f`.

Folds

`sum`, `product` en `and` lijken sterk op elkaar: een waarde voor `[]` en voor `x:xs` combineer je `x` met het resultaat voor de rest. Abstraheer die twee bewegende delen naar de argumenten `f` en `v`:

Code:

```haskell
foldr :: (a → b → b) → b → [a] → b
foldr _ v []     = v
foldr f v (x:xs) = f x (foldr f v xs)

sum     = foldr (+) 0
product = foldr (*) 1
and     = foldr (&&) True
length  = foldr (\_ y → 1 + y) 0
```

Wat gebeurt er:

```
foldr (+) 0 [x,y,z]
= foldr (+) 0 (x:(y:(z:[])))
= x + (y + (z + 0))

(:) wordt vervangen door (+)
[]  wordt vervangen door 0
```

`foldr` zet haakjes naar **rechts**. Het spiegelbeeld is `foldl`, met de haakjes naar links en een accumulator als tweede argument:

Code:

```haskell
foldl :: (b → a → b) → b → [a] → b
foldl _ v []     = v
foldl f v (x:xs) = foldl f (f v x) xs

-- foldl (+) 0 [x,y,z]
-- = (((0 + x) + y) + z)
```

Niet altijd gelijk:

```
> foldr (-) 0 [1,2,3]
2
> foldl (-) 0 [1,2,3]
-6
```

`foldr` en `foldl` geven hetzelfde resultaat (voor eindige lijsten) als de operatie f **associatief** is en de beginwaarde v een **identiteit** ($f\ v\ x = x = f\ x\ v$). Zo'n structuur heet een monoid, zoals $(+, 0)$.

Goede stijl is om expliciete recursie te vermijden en `map`, `filter` en folds te gebruiken: de bedoeling is duidelijker, er is minder te debuggen en terminatie is gegarandeerd. Bijvoorbeeld `count p = length . filter p`. En `applyAll` als fold:

```haskell
applyAll fs x = foldl (\y f → f y) x fs
applyAll      = foldr (flip (.)) id     -- let op: foldr (.) id is fout (volgorde!)
```

Voorbeeld **(Type inferentie van adders)**:

`adders = map (\n → \x → n + x)`, na eta-reductie gewoon `map (+)`. Specialiseer naar `Int`:
$(+) :: Int \to (Int \to Int)$ en $map :: (a \to b) \to [a] \to [b]$. Dus $a \to b = Int \to (Int \to Int)$, dus $a = Int$ en

$b = Int \to Int$. Algemeen: `adders :: Num a ⇒ [a] → [a → a]`. De uitkomst is een lijst van functies: `[a 5 | a ← adders [1,2,3]] = [6,7,8]`.

5 | Purely functional data structures

Persistent versus ephemeral

Hoe ziet `x:xs` eruit in het geheugen? Je maakt maar **één nieuwe cel** die naar `x` en naar de bestaande `xs` wijst. `drop 2 xs` is gewoon een pointer verder in dezelfde lijst. En `l ++ xs` kopieert `l`, maar `xs` wordt **gedeeld**. Oude versies blijven dus bestaan.
- **Persistent** (puur functioneel): oude versies blijven beschikbaar.
- **Ephemeral** (traditioneel): een update overschrijft de oude versie.

Voordelen van persistent: je hebt makkelijk oud en nieuw tegelijk, je kan deelberekeningen onafhankelijk doen en de output mag oude versies bevatten (bijvoorbeeld tails).

Successor datastructuur

Opgave: bewaar een verzameling S van geordende elementen zodat je snel de *successor* van een query q vindt: het kleinste element in S dat groter of gelijk is aan q. Bijvoorbeeld $S = \{1, 4, 5, 8, 9, 20\}$, de successor van 6 is 8. Het type is `lookupGE :: Ord a ⇒ a → SuccDS a → Maybe a`.
- Ongeordende lijst: $O(n)$.
- Geordende lijst: nog steeds $O(n)$.
- Binary search tree (BST): $O(\log n)$ *als* de boom gebalanceerd is.

BST + queries:

```
data Tree a = Leaf
            | Node (Tree a) a (Tree a)

lookupGE q Leaf = Nothing
lookupGE q (Node l x r)
  | x < q     = lookupGE q r
  | otherwise = case lookupGE q l of
      Nothing → Just x
      Just sq → Just sq
```

Insert (bouwt een nieuwe boom!):

```
insert :: Ord a ⇒ a → Tree a → Tree a
insert x Leaf = Node Leaf x Leaf
insert x t@(Node l y r)
  | x < y     = Node (insert x l) y r
  | x == y    = t
  | otherwise = Node l y (insert x r)

-- foldr insert Leaf [1..5] geeft een
-- scheve boom (eigenlijk een lijst)
```

Een andere vraag is hoe je een BST test. `isBST (Node l x r) = all (<= x) (elems l) && all (>= x) (elems r) && isBST l && isBST r` werkt, maar kost $O(n^2)$ (oefening: $O(n)$). Een gebalanceerde boom uit een gesorteerde lijst bouw je door het midden als wortel te nemen (`buildBalanced`, $O(n \log n)$).

Red-black trees

Een zelfbalancerende boom met $O(\log n)$ queries en updates. Eigenschappen:
1. leaves zijn zwart
2. de root is zwart
3. rode nodes hebben zwarte kinderen
4. voor elke node hebben beide kinderen dezelfde *blackheight* (aantal zwarte nodes op elk pad naar de leaves)

```
data Color  = Red | Black deriving (Show, Eq)
data RBTree a = Leaf
              | Node Color (RBTree a) a (RBTree a)
              deriving (Show, Eq)
```

Het type dwingt alleen eigenschap 1 af. De insert-strategie: vervang een zwarte leaf door een rode node (de blackheight blijft gelijk). Het enige probleem is dan twee rode nodes onder elkaar. Die los je op terwijl je terug omhoog loopt met `balance`, en aan het eind kleur je de root zwart.

```haskell
insert :: Ord a ⇒ a → RBTree a → RBTree a
insert x = blackenRoot . insert' x

insert' x Leaf = Node Red Leaf x Leaf
insert' x t@(Node c l y r)
  | x < y     = balance c (insert' x l) y r
  | x == y    = t
  | otherwise = balance c l y (insert' x r)

blackenRoot Leaf = Leaf
blackenRoot (Node _ l y r) = Node Black l y r
```

Er zijn maar vier configuraties met twee rode nodes bij de root, en alle vier worden dezelfde boom (root rood, kinderen zwart):

```haskell
balance :: Color → RBTree a → a → RBTree a → RBTree a
balance Black (Node Red (Node Red a x b) y c) z d = Node Red (Node Black a x b) y (Node Black c z d
    )
balance Black (Node Red a x (Node Red b y c)) z d = Node Red (Node Black a x b) y (Node Black c z d
    )
balance Black a x (Node Red (Node Red b y c) z d) = Node Red (Node Black a x b) y (Node Black c z d
    )
balance Black a x (Node Red b y (Node Red c z d)) = Node Red (Node Black a x b) y (Node Black c z d
    )
balance c l x r = Node c l x r
```

In de standaardbibliotheek zit dit al: `Data.Set` (self-balancing BST), `Data.Map` (met extra info, `lookup :: Ord k ⇒ k → Map k v → Maybe v`) en `Data.Sequence` (snel voor en achter). Allemaal persistent.

Voorbeeld **(Toepassing: point location (Mario))**:

Welk platform ligt direct onder Mario op (x, y)? Als je de platforms die de verticale lijn op x snijden in een `Set` hebt, is het de successor van y. Als de lijn naar rechts schuift, voeg je platforms toe of haal je ze weg. Omdat `Set` persistent is blijven de oude versies bestaan: bewaar ze in een `Map`. Een query is dan: zoek de versie op tijd x (successor query), en daarin de successor van y.

6 | Case studies

Doel: een probleem opdelen in deelproblemen, de oplossingen samenstellen, en het verschil zien tussen expressies **interpreteren** (syntax → waarde) en **transformeren** (syntax → nieuwe syntax).

Proposities

Een propositie is opgebouwd uit `True`/`False`, variabelen, negatie, conjunctie, disjunctie en implicatie. Een *tautologie* is waar voor elke toekenning van waarden aan de variabelen, bijvoorbeeld $X \vee \neg X \vee \neg Y$. Het plan: (1) een datatype `Prop`, (2) een functie `tv` die de waarheidswaarde berekent, (3) alle toekenningen genereren, (4) `taut`.

```haskell
data Prop = Basic Bool | Var Char | Not Prop
          | Prop :/\: Prop | Prop :\/: Prop | Prop :=>: Prop
          deriving Show

-- (X /\ Y) => not Y
example = (Var 'X' :/\: Var 'Y') :=>: (Not (Var 'Y'))

type Assignment = Map Char Bool

tv :: Assignment → Prop → Bool
tv _ (Basic b)    = b
tv m (Var v)      = case lookup v m of
                      Nothing → error "Variable unknown!"
                      Just b  → b
```

```
tv m (Not p)      = not (tv m p)
tv m (p1 :/\: p2) = tv m p1 && tv m p2
tv m (p1 :\/: p2) = tv m p1 || tv m p2
tv m (p1 :=>: p2) = not (tv m p1) || tv m p2
```

Voor de toekenningen verzamel je eerst de variabelen (`vars`, zonder dubbelen met `nub`) en genereer je dan alle combinaties. Let op: voor nul variabelen is er **één** toekenning (`[empty]`), niet nul. Anders is `taut` altijd waar.

```
assigns :: Prop → [Assignment]
assigns = assigns' . vars

assigns' []     = [empty]
assigns' (v:vs) = [insert v True  as | as ← assigns' vs]
               ++ [insert v False as | as ← assigns' vs]

taut :: Prop → Bool
taut p = and [tv as p | as ← assigns p]
-- of: taut p = all (flip tv p) (assigns p)
```

Simplificatie: elke propositie is te schrijven met alleen $\neg$ en $\wedge$, met De Morgan ($A \vee B \equiv \neg(\neg A \wedge \neg B)$), dubbele negatie ($\neg\neg A \equiv A$) en $A \Rightarrow B \equiv \neg A \vee B$.

```
simp :: Prop → Prop
simp b@(Basic _) = b
simp v@(Var _)   = v
simp (Not p)     = case simp p of
                     Not q → q          -- dubbele negatie
                     q     → Not q
simp (p1 :/\: p2) = simp p1 :/\: simp p2
simp (p1 :\/: p2) = simp (Not (Not p1 :/\: Not p2))
simp (p1 :=>: p2) = simp (Not p1 :\/: p2)
```

Rekenkundige expressies

Hier scheiden we de *naam van de operatie* van de structuur van de expressie.

```
data ArithOp   = Plus | Minus | Times | Div deriving Show
data ArithExpr = Constant Integer
               | Variable Char
               | Op ArithOp ArithExpr ArithExpr
               deriving Show

eval :: Map Char Double → ArithExpr → Double
eval _ (Constant c) = fromInteger c
eval m (Variable v) = case lookup v m of
                        Nothing → error "unknown variable!"
                        Just x  → x
eval m (Op o x y)   = evalOp o (eval m x) (eval m y)
  where evalOp Plus  = (+)
        evalOp Minus = (-)
        evalOp Times = (*)
        evalOp Div   = (/)     -- let op: evalOp geeft een functie terug
```

Differentiatie bouwt syntax voor de afgeleide, er zijn geen waarden nodig (symbolic manipulation). Regels: $\frac{dx}{dx} = 1$, $\frac{dc}{dx} = 0$, $\frac{d(f \pm g)}{dx} = \frac{df}{dx} \pm \frac{dg}{dx}$, productregel $\frac{d(f \cdot g)}{dx} = \frac{df}{dx} \cdot g + f \cdot \frac{dg}{dx}$ en de quotiëntregel.

```
diff :: ArithExpr → Char → ArithExpr
diff (Constant _) _ = Constant 0
diff (Variable v) x | v == x    = Constant 1
                    | otherwise = Constant 0
diff (Op Plus  f g) x = Op Plus  (diff f x) (diff g x)
diff (Op Minus f g) x = Op Minus (diff f x) (diff g x)
```

```haskell
diff (Op Times f g) x = Op Plus (Op Times (diff f x) g)
                                (Op Times f (diff g x))
diff (Op Div f g) x   = Op Div (Op Minus (Op Times (diff f x) g)
                                         (Op Times f (diff g x)))
                               (Op Times g g)
```

Hetzelfde patroon zie je in compilers: broncode als boom, die je analyseert of transformeert.

7 | Software design en project management

Modules en packages

Haskell organiseert code in modules: één module per bestand, één concept per module, en de bestandsnaam komt overeen met de modulenaam (`My.Long.Prefix.MyModule` staat in `My/Long/Prefix/MyModule.hs`).

Importeren:

```haskell
import Data.List
import Data.List (nub, permutations)
import Data.List hiding (nub)
import qualified Data.List as L
-- dan: L.nub, L.permutations
```

Exporteren:

```haskell
module MyModule
  ( thing1, thing2
  , Foo(..)    -- constructors zichtbaar
  , Bar        -- abstract
  ) where
```

Een *package* is de eenheid van distributie (te vinden op Hackage, bijvoorbeeld `containers` of `gloss`). Je beschrijft je project in een `.cabal` bestand: algemene info, en per `executable` een `main-is` (het startpunt, een module `Main` met `main :: IO ()`), `hs-source-dirs` en `build-depends` (bijvoorbeeld `base, transformers >= 0.5 && < 1.0`). Met Cabal (of Stack) hoef je zelf `ghc` niet aan te roepen:

```
$ cabal update          # lijst met packages bijwerken
$ cabal build           # bouwen (installeert dependencies)
$ cabal run your-executable
```

Software design

- **Scheid puur en impure**: (1) impure deel haalt input op, (2) puur deel verwerkt de data, (3) impure deel geeft het resultaat. Impure functies zijn gemarkeerd met `IO`.
- **Model View Controller**: `data Model = ...` (alle state), `view :: Model → Picture` en `update :: Input → Model → Model`.

Maak onmogelijke states onmogelijk. Gebruik niet `type Boolean = Int` (0/1) maar `data Boolean = False | True`. Gebruik ook geen type waar sommige velden bij sommige constructors betekenisloos zijn:

```haskell
data Object = Ship ... | Player { score :: Int, numLives :: Int } | Enemy | Wall | Empty

getNumLives :: Object → Int
getNumLives Wall = ?      -- slechte signature, partial function ⇒ runtime error
```

Eén type per concept, ook al zijn types isomorf. Dit voorkomt "Boolean blindness" en het verwisselen van dingen:

```haskell
data PlayerStatus = Alive | Dead
data LevelStatus  = Finished | InProgress
computeScore :: PlayerStatus → LevelStatus → Int    -- i.p.v. Bool → Bool → Int

data Point  = Point Float Float
data Vector = Vector Float Float
translate :: Point → Vector → Point
```

Type classes voor abstracties, bijvoorbeeld `class HasPosition a where getPosition :: a → Point; move :: a → Vector → a` of `class CanRender a where render :: a → Picture`.

Modules voor abstractie: gebruik modules om invarianten te bewaren. Exporteer het type *abstract* (zonder constructors) en geef alleen functies:

```haskell
module Name (Name, mkName, render) where
import Data.Char

newtype Name = MkName String deriving Eq

mkName :: String → Name      -- altijd hoofdletter + rest kleine letters
mkName []     = MkName []
mkName (c:cs) = MkName $ toUpper c : map toLower cs

render :: Name → String
render (MkName s) = s
```

Een *exposed* type (`Type(..)`) laat de buitenwereld constructors en pattern matching gebruiken. Een *abstract* type (`Type`) niet, waardoor je de implementatie kan veranderen zonder afhankelijke code te herschrijven (decoupling).

8 | Input en output

Purity en side effects

Haskell is **referentieel transparant**: een expressie vervangen door een gelijke expressie verandert de betekenis niet. Inlinen (`let x = e in ... x ... x ...` is gelijk aan `... e ... e ...`), dupliceren, weggooien en herordenen zijn dus veilig. Dit geeft de compiler veel vrijheid (herordenen, lazy evaluation, parallellisme). Dit werkt **niet** bij side effects:

```haskell
-- hypothetisch (niet Haskell): getChar :: Char leest het volgende teken
let k = getChar in k == k      -- leest 1x, dus True
getChar == getChar             -- leest 2x, kan False zijn
```

Daarom heeft getChar in Haskell het type `IO Char`. Acties met side effects (scherm, random generator, netwerk, database) zijn gewone waardes die een actie *beschrijven*.

```haskell
getChar  :: IO Char          putChar   :: Char → IO ()
getLine  :: IO String        putStr    :: String → IO ()
getArgs  :: IO [String]      putStrLn  :: String → IO ()
return   :: a → IO a
(>>=)    :: IO a → (a → IO b) → IO b
```

Een model van IO

Conceptueel (de details zijn verborgen) is een actie een functie die de wereld verandert: `type IO a = World → (a, World)`. Output geeft `()` terug, input geeft de waarde *en* de nieuwe wereld terug. Omdat `putChar . getChar` niet typecheckt, hebben we **bind** nodig, die de wereld doorgeeft:

```haskell
(>>=) :: IO a → (a → IO b) → IO b
(f >>= g) w = g a' w' where (a', w') = f w

return :: a → IO a
return a = \w → (a, w)       -- geen "return" zoals in C#, breekt de flow niet!

echo      = getChar >>= \c → putChar c
getUpper  = getChar >>= \c → return (toUpper c)
```

Er is geen veilige `IO a → a`: zo blijft de buitenwereld buiten pure expressies en blijft de referentiële transparantie behouden. `return` embed alleen een pure waarde in IO.

do-notatie

Werken met `>>=` is omslachtig, dus is er `do`. Een `do`-blok wordt vertaald naar geneste `>>=`, en `←` geeft een naam aan het *resultaat* van een actie.

do-notatie:

```
getLine = do
  c ← getChar
  case c of
    '\n' → return []
    _    → do rest ← getLine
              return (c : rest)
```

Vertaling:

```
do x1 ← a1
   x2 ← a2
   expr
-- wordt
a1 >>= (\x1 →
a2 >>= (\x2 →
expr))
```

Voorbeeld **(Guess a number)**:

Binary search: raad het midden van het resterende interval.

```
guess :: Int → Int → IO ()
guess l u
  | l > u     = putStrLn "No number fits those answers."
  | otherwise = do
      let m = l + ((u - l) `div` 2)
      putStr ("Is it " ++ show m ++ "? ")
      putStrLn "(g = greater, l = less, c = correct)"
      k ← getLine
      case k of
        "g" → guess (m + 1) u
        "l" → guess l (m - 1)
        "c" → putStrLn "Guessed"
        _   → do putStrLn "Please type g/l/c!"
                 guess l u

main :: IO ()
main = do (l:u:_) ← getArgs
          guess (read l) (read u)
```

Bestanden: `readFile :: FilePath → IO String` en `writeFile :: FilePath → String → IO ()`, samen met `lines`, `unlines`, `show` en `read`.

IO als first-class citizen

IO-acties kan je in lijsten stoppen en doorgeven. Maar **bouwen** is niet hetzelfde als **uitvoeren**: `map (\name → putStrLn ("Hello, " ++ name)) ["Mary", "John"] :: [IO ()]` bouwt alleen een lijst van acties. Om ze uit te voeren moet je ze sequencen in een actie die uiteindelijk door `main` wordt uitgevoerd.

```
sequence_ :: [IO a] → IO ()           -- voer uit, gooi resultaten weg
sequence_ []     = return ()
sequence_ (a:as) = do a
                      sequence_ as

sequence :: [IO a] → IO [a]           -- voer uit, bewaar resultaten
sequence []     = return []
sequence (a:as) = do r  ← a
                     rs ← sequence as
                     return (r:rs)

mapM_ :: (a → IO b) → [a] → IO ()   -- sequence_ . map
forM_ = flip mapM_                    -- "imperatief" ogende code
greet names = forM_ names $ \name → putStrLn ("Hello, " ++ name)

liftM2 f ia ib = do { a ← ia; b ← ib; return (f a b) }
```

Naamconventie: een functie die eindigt op _ gooit informatie weg (`mapM` versus `mapM_`).

Randomness

`System.Random` is puur: dezelfde generator-state geeft hetzelfde resultaat. Je moet de state dus **doorgeven**:

```
generateTwoNumbers :: RandomGen g ⇒ g → ((Int, Int), g)
generateTwoNumbers g = let (v1, g1) = random g
                           (v2, g2) = random g1   -- gebruik de nieuwe state!
                       in ((v1, v2), g2)
```

Voor een verse generator: `getStdRandom :: (StdGen → (a, StdGen)) → IO a`, met `randomRIO = getStdRandom . randomR` en `randomIO = getStdRandom random`.

9 | Functors en monads

Functors

`map` voor lijsten, `mapMay` voor `Maybe` en `mapTree` voor bomen hebben hetzelfde patroon. Het verschil zit in de *type constructor* (`[]`, `Maybe`, `Tree`). Een type constructor met een nette "map" heet een **functor**. Dit is *higher-kinded abstraction*: `f` is zelf een type constructor (`Maybe` past, `Maybe Int` niet).

```
class Functor f where
  fmap :: (a → b) → f a → f b

instance Functor Maybe where
  fmap _ Nothing  = Nothing
  fmap f (Just x) = Just (f x)

instance Functor ((→) r) where     -- functies: map is compositie!
  fmap ab ra = \r → ab (ra r)

instance Functor IO where
  fmap f a = do x ← a
                return (f x)

-- (<$>) = fmap, dus:  toUpper <$> getChar
inc :: Functor f ⇒ f Int → f Int
inc xs = (+1) <$> xs
```

Wetten voor een geldige functor: `fmap id = id` en `fmap (f . g) = fmap f . fmap g`. Een lijst-instance die ook omkeert (`fmap id [1,2] = [2,1]`) breekt de identiteitswet. Een instance die altijd `[]` teruggeeft breekt ook de identiteitswet (maar voldoet wel aan de compositiewet).

Monads

Neem de evaluatie van `ArithExpr` met `Maybe`. Zonder abstractie krijg je diep geneste `case`-expressies die steeds hetzelfde doen: bij `Nothing` stoppen, bij `Just x` verder met `x`. Pak dat patroon in een higher-order functie:

Voor:

```
eval m (Op Plus x y)
  = case eval m x of
      Nothing → Nothing
      Just x' → case eval m y of
        Nothing → Nothing
        Just y' → Just (x' + y')
```

Na (met do):

```
eval m (Op Plus x y) = do
  x' ← eval m x
  y' ← eval m y
  return (x' + y')

validate = do nm' ← validateName nm
              ag' ← validateAge ag
              return (Record nm' ag')
```

Dit is hetzelfde als `>>=` voor IO. Een **monad** is een type constructor met `return` en `>>=` (en elke monad is ook een functor):

```
class Functor m ⇒ Monad m where
  return :: a → m a
  (>>=)  :: m a → (a → m b) → m b

instance Monad Maybe where
  return = Just
  Nothing >>= _ = Nothing
  Just x  >>= f = f x
```

`do`-notatie werkt voor elke monad, maar een `do`-blok werkt maar met **één** monad. Denk ook aan: `return` onderbreekt niets, dus `do x ← m; return 3; return (x + 1)` telt gewoon 1 op bij de waarde in `m`. Waarom is `fmap step mx` geen vervanger voor `mx >>= step` met `step :: a → Maybe b`? Het type is `Maybe (Maybe b)`. Voor `mx = Just v` en `step v = Nothing` geeft `fmap Just Nothing` en `>>= Nothing`.

De list monad

`return x = [x]` en `xs >>= f = concat (map f xs)`. De list monad modelleert **zoeken** en **non-determinisme**: elke variabele heeft meerdere mogelijke waardes.

Code:

```
do x ← [1,2,3]
   y ← [4,5,6]
   return (x + y)

-- Een mislukte tak is []:
guard :: Bool → [()]
guard True  = [()]
guard False = []

sumTriples xs = do
  x ← xs; y ← xs; z ← xs
  guard (x + y == z)
  return (x,y,z)
```

Output:

```
[5,6,7,6,7,8,7,8,9]

> sumTriples [1,2,3]
[(1,1,2),(1,2,3),(2,1,3)]
```

Voor lijsten betekent dit hetzelfde als `[(x,y,z) | x ← xs, y ← xs, z ← xs, x + y == z]`. Je combineert oplossingen met `++`, bijvoorbeeld `triples xs = sumTriples xs ++ pytTriples xs` (een disjunctie). Monads met zo'n "falen" en "combineren" zijn `MonadPlus` (`mzero`, `mplus`); voor `Maybe` is `mzero = Nothing` en `mplus` pakt de eerste `Just`.

10 | Wetten en inductie

Wetten en equational reasoning

Een **wet** is een vergelijking die geldt voor alle waardes van de genoemde types, bijvoorbeeld $x + y = y + x$ of in logica $\neg(x \wedge y) = \neg x \vee \neg y$. Waarom wil je programma-equivalenties? Voor *performance* (vervang een dure expressie door een goedkopere gelijke) en voor *correctheid* (de implementatie heeft de bedoelde eigenschap). Omdat Haskell referentieel transparant is, is elke definitie ook een herschrijfregel (in beide richtingen).

Belangrijke wetten:

```
f . (g . h) = (f . g) . h                    -- compositie is associatief
map f (xs ++ ys) = map f xs ++ map f ys       -- map verdeelt over ++
map (f . g) = map f . map g                   -- fusion (1 traversal i.p.v. 2)
foldr op e xs = foldl op e xs                 -- als op associatief en e de identiteit is
```

Bewijs altijd met een hint bij elke stap. Bijvoorbeeld `foldr op e [x] = x` als e een rechtse eenheid is:

```
foldr op e [x]
```

```
= -- lijstnotatie
foldr op e (x : [])
= -- definitie foldr, cons-geval
op x (foldr op e [])
= -- definitie foldr, lege lijst
op x e
= -- e is een rechtse eenheid van op
x
```

Welk bewijsprincipe?

- **Functies gelijk bewijzen**: extensionaliteit. Bewijs $f = g$ door voor een *willekeurige* x te bewijzen dat $f\ x = g\ x$.
- **Niet-recursieve data**: case distinction (alle constructors afgaan, bijvoorbeeld `True` en `False`).
- **Recursieve data**: structurele inductie.

Voorbeeld **(not is een involutie (extensionaliteit + cases))**:

`not . not = id`. Neem een willekeurige `x`. Geval `False`:
`(not . not) False = not (not False) = False = id False`. Geval `True`: analoog.

Structurele inductie

Stappen: (1) formuleer de wet en kies de inductievariabele (het argument dat de recursieve definities consumeren), (2) bewijs de basisgevallen direct, (3) stel voor elk recursief veld een inductiehypothese (IH) op, (4) vouw definities uit tot de IH toepasbaar is. Voor lijsten: bewijs het voor `[]`, en neem aan dat het geldt voor `xs` om het te bewijzen voor `z:zs`.

Voorbeeld: `map f (xs ++ ys) = map f xs ++ map f ys`, inductie op `xs` (want `++` consumeert zijn eerste argument).

```
-- Geval xs = []
map f ([] ++ ys) = map f ys                  -- def. (++)
map f [] ++ map f ys = [] ++ map f ys = map f ys   -- def. map, def. (++)

-- Geval xs = z:zs, IH: map f (zs ++ ys) = map f zs ++ map f ys
map f ((z:zs) ++ ys)
= map f (z : (zs ++ ys))                     -- def. (++)
= f z : map f (zs ++ ys)                     -- def. map
= f z : (map f zs ++ map f ys)               -- IH

map f (z:zs) ++ map f ys
= (f z : map f zs) ++ map f ys               -- def. map
= f z : (map f zs ++ map f ys)               -- def. (++)
```

Soms loop je vast en heb je een **lemma** nodig. Bij `reverse (reverse xs) = xs` verschijnt `reverse (reverse zs ++ [z])`, en daarvoor heb je `reverse (xs ++ ys) = reverse ys ++ reverse xs` nodig (zelf te bewijzen met inductie, met `ws ++ [] = ws` en associativiteit van `++`) en `reverse [x] = [x]`.

Hetzelfde geldt voor getallen (Peano) en bomen: één IH per recursief veld.

Peano:

```
data Nat = Zero | Succ Nat

add Zero     m = m
add (Succ n) m = Succ (add n m)

-- Bewijs: add n Zero = n, inductie op n
-- Zero:    add Zero Zero = Zero
-- Succ p:  add (Succ p) Zero
--   = Succ (add p Zero)   -- def. add
--   = Succ p              -- IH
```

Bomen:

```
size Leaf = 0
size (Node l _ r) = 1 + size l + size r
mirror Leaf = Leaf
mirror (Node l x r) =
  Node (mirror r) x (mirror l)

-- size (mirror t) = size t
-- IH1: size (mirror l) = size l
-- IH2: size (mirror r) = size r
-- = 1 + size (mirror r) + size (mirror l)
-- = 1 + size r + size l      -- IH1, IH2
```

```
12 -- = 1 + size l + size r       -- (+) comm.
```

Als je vastloopt: zoek een sterkere stelling of een hulp-lemma. Inductieve bewijzen komen zeker op het tentamen (lijsten, natuurlijke getallen, bomen of een ander recursief type).

11 | Lazy evaluation

Evaluatiestrategieën

Hoe kom je van `square (1 + 2)` (met `square x = x * x`) aan 9?
- **Eager / call-by-value**: eerst de argumenten uitrekenen (`square 3`), dan de functie. Zo werken de meeste talen.
- **Call-by-name**: het argument as-is in de body zetten, `(1 + 2) * (1 + 2)`. Dat rekent `1 + 2` twee keer uit.
- **Lazy (Haskell)**: call-by-name, maar identieke expressies worden **gedeeld** en pas uitgerekend *als het nodig is*. Een niet-uitgerekende expressie heet een **thunk**. Lazy doet nooit meer stappen dan call-by-value.

Maakt het uit? Volgens de stelling van Church-Rosser geeft elke strategie dezelfde waarde voor **terminerende** programma's. Maar niet bij oneindige loops en exceptions, en de performance kan verschillen. Lazy termineert vaker dan eager:

Code:

```
1 loop x = loop x     -- termineert nooit
2 const x y = x
3
4 const 5 (loop 3)
```

Eager versus lazy:

```
eager: blijft hangen in loop 3
lazy:  5
```

Waarom is lazy handig?

- **Short-circuiting**: `False && _ = False`, dus `False && loop True` termineert.
- **Eigen control structures**: `if_ True t _ = t; if_ False _ e = e` evalueert maar één tak. In eager talen moet `if` ingebouwd zijn.
- **Separation of concerns**: bereken alleen wat je gebruikt, bijvoorbeeld `minimum' d = fst . minAndMax d`.
- **Oneindige structuren**, zolang je maar een eindig stuk gebruikt.

Code:

```
1 ones = 1 : ones
2 nats = 0 : map (+1) nats
3
4 fibs = 0 : 1 : zipWith (+) fibs (tail fibs)
5 fib n = fibs !! n
6
7 primes = sieve [2 ..]
8 sieve (p:ns) =
9   p : sieve [n | n ← ns, n `mod` p /= 0]
```

Output:

```
> take 2 ones
[1,1]
> take 5 fibs
[0,1,1,2,3]
> take 5 primes
[2,3,5,7,11]
```

Weak Head Normal Form

Hoeveel wordt er uitgerekend? Precies genoeg om te kiezen welke branch je volgt (pattern matching of een guard). Een expressie is in **WHNF** als het is: een constructor met (mogelijk niet-uitgerekende) data erin (`Just (1 + 2)`), een anonieme functie (`\x → x + 1`), of een functie met te weinig argumenten (`map minimum`). Bijvoorbeeld `(x + 1) : foldr (:) [] [1..5]` is al in WHNF, `map (\x → x + 1) [1..5]` niet.

Een functie is **strict** in een argument als $f \perp = \perp$ (non-terminatie in dat argument geeft non-terminatie als resultaat). `(+)` is strict in beide, `const` alleen in zijn eerste argument. Enkele voorbeelden: `(\x → 0) undefined = 0`, `(\x → undefined) 0 = undefined`, `length (map undefined [1,2]) = 2`.

Lazy en performance

`foldl (+) 0 [1,2,3]` bouwt `((0 + 1) + 2) + 3` op in thunks. Dat kost geheugen: een **space leak** (data die groter wordt of langer leeft dan verwacht, wat meer garbage collection en dus slechtere performance geeft). Oplossing: forceer de optelling vooraf met `seq`.

```
seq :: a → b → b           -- rekent eerst x uit tot WHNF, daarna y
foldl' _ v []     = v
foldl' f v (x:xs) = let z = f v x in z `seq` foldl' f z xs

($!) :: (a → b) → a → b    -- strict application
f $! x = x `seq` f x
```

`seq` rekent maar uit tot WHNF: `seq (undefined, undefined) 0 = 0` en `seq (1 : undefined) 0 = 0`, maar `(\x → 0) $! undefined = undefined`.

Lokale memoization (dynamic programming): de naïeve `fib` roept dezelfde deelproblemen steeds opnieuw aan. Bewaar de deelresultaten in een lijst, sharing zorgt dat elk maar één keer wordt uitgerekend (nog sneller met bijv. `IntMap`):

```
memo_fib n = go n
  where go i = fibs !! i
        fibs = map fib [0 ..]
        fib 0 = 0
        fib 1 = 1
        fib n = go (n-1) + go (n-2)
```

12 | Property testing met QuickCheck

Testen geeft vertrouwen in de correctheid, maar kan de afwezigheid van bugs niet bewijzen. **QuickCheck** beschrijft eigenschappen (properties) als Haskell-programma's en genereert automatisch random testcases op basis van het datatype.

Case study: insertion sort

Een goede specificatie is zo precies als nodig, maar niet preciezer, en dwingt geen specifiek algoritme af. Een eerste poging: sorteren verandert de lengte niet. Dit vindt de eerste bug (`x : ys` moest `x : y : ys` zijn).

Code:

```
sort []     = []
sort (x:xs) = insert x xs  -- bug 2!

insert x [] = [x]
insert x (y:ys)
  | x <= y    = x : ys     -- bug 1!
  | otherwise = y : insert x ys

sortPreservesLength xs =
  length (sort xs) == length xs
```

Output:

```
> quickCheck sortPreservesLength
Failed! Falsifiable, after 4 tests:
[0,3]

-- na fix 1:
OK, passed 100 tests.
```

Maar `id` voldoet hier ook aan, dus de specificatie moet scherper. Een gesorteerde lijst test je met `sorted`. Let op: een eerste versie met `x < y` was zelf fout (dubbelen!), de specificatie kan ook bugs hebben. Bug 2 is dat `sort` de staart niet recursief sorteert: `insert x (sort xs)`. Maar lengte + gesorteerd is nog niet genoeg, want `evilNoSort xs = replicate (length xs) 1` voldoet ook. Daarom:

```
sorted (x:y:ys) = x <= y && sorted (y:ys)
sorted _        = True

permutes f xs = f xs `elem` permutations xs

-- minimale volledige specificatie:
```

```
sorts alg xs = and [ alg `permutes` xs, sorted (alg xs) ]
```

QuickCheck in het algemeen

`quickCheck :: Testable prop ⇒ prop → IO ()`. Properties zijn meestal functies (met willekeurig veel argumenten) die een `Bool` teruggeven, waarbij de argumenttypes `Arbitrary` en `Show` moeten zijn. Ook zonder argumenten kan het (QuickCheck bevat unit tests).

```
appendLength xs ys = length xs + length ys == length (xs ++ ys)
takeDrop n xs      = take n xs ++ drop n xs == xs
dropTwice m n xs   = drop m (drop n xs) == drop (m + n) xs
-- dropTwice: Falsifiable, bijv. m = 1, n = -1, xs = [0] (negatieve getallen!)
```

Met `collect` laat je de verdeling van de testdata zien (waarom is dat belangrijk?). **Implicaties** gebruik je als een eigenschap alleen voor bepaalde input geldt. Met `implies` (`not x || y`) lijkt de test te slagen terwijl 88% van de gevallen niets test. Gebruik `==>`: slechte gevallen worden weggegooid.

```
insertPreservesOrdered x xs = sorted xs ==> sorted (insert x xs)
-- Arguments exhausted after 20 tests: als je ook length xs > 2 eist, is bijna
-- geen random lijst gesorteerd.
```

Eigen generators

`Gen a` is een abstract type (alleen toegang tot random getallen). Standaard definieer je `instance Arbitrary a where arbitrary :: Gen a`. Combinators: `choose`, `oneof`, `frequency`, `elements`, `sized`.

```
instance Arbitrary Dir where
  arbitrary = elements [North, East, South, West]

-- gesorteerde lijsten genereren: pas de standaard generator aan
mkSorted []       = []
mkSorted [x]      = [x]
mkSorted (x:y:ys) = x : mkSorted ((x + abs y) : ys)    -- mkSorted [1,2,-3,4] = [1,3,6,10]

genSorted :: Gen [Int]
genSorted = mkSorted <$> arbitrary

insertPreservesOrdered x = forAll genSorted (\xs → sorted (insert x xs))

-- of met een newtype:
newtype SortedIntList = SortedIntList [Int]
instance Arbitrary SortedIntList where
  arbitrary = fmap SortedIntList genSorted
```

Shrinking (`shrink :: a → [a]`) maakt een falende testcase steeds structureel kleiner ([2,3] is kleiner dan [1,2,3]) tot er geen kleinere falende meer is.

Verder dan testen

Bewijzen kan met equational reasoning (veel handwerk, lazy en exceptions worden niet gedekt), met interactive theorem provers (Coq, Isabelle/HOL) of met expressievere types: *dependent types* (Agda, Idris, Coq: `append :: List n a → List m a → List (n + m) a`) en *refinement types* (LiquidHaskell). Ook geven types soms *theorems for free*: `f :: a → a` kan alleen de identiteit zijn en `g :: (a, b) → (b, a)` alleen de swap.

13 | State monad en applicatives

De State monad

`State s a` levert een resultaat van type `a` en een nieuwe state van type `s` op, altijd als paar `(resultaat, state)`. De operaties zijn `get :: State s s`, `put :: s → State s ()`, `modify :: (s → s) → State s ()` en `runState :: State s a → s → (a, s)`.

Case study: RPN-calculator. In Reverse Polish Notation staat de operator achter zijn operanden (`3 4 + 2 *` is 14), dus haakjes zijn niet nodig. Een stack machine evalueert dit instructie voor instructie: getallen gaan op de stack, operaties pakken de bovenste elementen.

```
data Instr = Number Float | Operation ArithOp
type RPN   = [Instr]
type Stack = [Float]

pop  :: Stack → (Float, Stack)
pop (x:xs) = (x, xs)
push :: Float → Stack → ((), Stack)
push x xs  = ((), x : xs)

evalInstr (Number f) s = push f s
evalInstr (Operation op) s =
  let (x, s1) = pop s       -- eerst gepopte waarde = rechter operand
      (y, s2) = pop s1
  in push (evalOp op y x) s2
```

Het patroon `let (x, newStack) = f oldStack in ...` komt steeds terug, dus pak het in een functie. Met een `newtype` (een type synoniem mag niet gedeeltelijk worden toegepast, dus kan geen `Monad`-instance krijgen) wordt het een monad:

```
newtype State s a = S (s → (a, s))

evalState :: State s a → s → a
evalState (S f) s = fst (f s)

instance Monad (State s) where
  return x = S $ \s → (x, s)          -- laat de state ongemoeid
  (S f) >>= g = S $ \s → let (x, s') = f s
                             S g'    = g x
                         in g' s'

evalInstr (Operation op) = do x ← pop
                              y ← pop
                              push (evalOp op y x)
```

Je kan een `State s a` zien als een "doos": stop er een state in, er komt een waarde en een nieuwe state uit. `>>=` verbindt twee dozen: de nieuwe state van de eerste gaat de tweede in.

Voorbeeld **(Boom labelen met een teller)**:

`label :: Tree a → Tree (Int, a)` nummert de nodes in-order (links, node, rechts). De state is de teller.

```
nextLabel :: State Int Int
nextLabel = S $ \i → (i, i + 1)

label t = evalState (label' t) 0
  where
    label' Leaf = return Leaf
    label' (Node l x r) = do l' ← label' l
                             i  ← nextLabel
                             r' ← label' r
                             return (Node l' (i, x) r')
```

Monad wetten

```haskell
return x >>= f = f x                          -- left identity
m >>= return   = m                            -- right identity
(m >>= f) >>= g = m >>= (\x → f x >>= g)   -- associativiteit (volgorde van effecten blijft)
```

Daarom moet `return` voor State de state ongemoeid laten: een versie die de state vervangt (`return x = \_ → (x, [])`) breekt left identity.

Overzicht van monads: `Maybe` (optionele waardes en falen), `State` (een impliciete waarde doorgeven), `[]` (zoeken, non-determinisme), `IO` (impure input/output). Er zijn er meer: `Either` (falen met reden), `Reader` (read-only omgeving), `Writer` (bijv. een log), `STM` (transacties) en `Cont` (non-local control flow).

Applicatives

`liftM2` (en `liftM3`, ...) til een functie met meerdere argumenten naar een monad, maar je hebt per aantal argumenten een andere versie nodig. Beter: `fmap f x` geeft bij `f :: a → b → c` het type `f (b → c)`, de functie zit nu *in* de functor. Je hebt dus `(<*>) :: f (b → c) → f b → f c` nodig. Dan kan je een functie met elk aantal argumenten tillen:

```haskell
f <$> ma <*> mb <*> ... <*> my :: m z

class Functor f ⇒ Applicative f where
  pure  :: a → f a                    -- 0-aire functies
  (<*>) :: f (a → b) → f a → f b

-- Elke monad is een applicative:
pure = return
mf <*> mx = do f ← mf
               x ← mx
               return (f x)

-- Voorbeelden
(+) <$> xs <*> ys                      -- = [x + y | x ← xs, y ← ys]
Person <$> validateFirstName fn <*> validateLastName ln

label' (Node l x r) = Node <$> label' l
                           <*> ((\i → (i, x)) <$> nextLabel)
                           <*> label' r
```

Er bestaan ook applicatives die geen monad zijn. De hiërarchie is `Functor` → `Applicative` → `Monad`:

```haskell
fmap          :: (a → b)    → f a → f b
(<*>)         :: f (a → b) → f a → f b
flip (>>=)    :: (a → f b) → f a → f b
```

Wanneer heb je bind nodig? Bij `>>=` kan een eerder resultaat bepalen welke berekening er daarna komt. Dat kan niet met alleen `<*>`:

```haskell
-- A: past in applicative stijl:  (,) <$> getLine <*> getLine
do x ← getLine
   y ← getLine
   return (x, y)

-- B: de tweede actie hangt af van x, dus bind is nodig
do x ← getLine
   if null x then return "" else getLine
```

Samengevat: functors tillen functies van 1 argument, applicatives tillen functies van meerdere argumenten, monads laten een eerder resultaat de volgende berekening kiezen.